# Problem Set 6

Hoa T. Vu

## Discrete Probability

**Problem 1: Card Draw.** A standard deck has 52 cards: 4 suits (clubs, diamonds, hearts, spades), each with 13 ranks (2–10, J, Q, K, A). Two cards are drawn *without replacement*.

(a) What is the probability that both cards are hearts?

(b) What is the probability that the two cards have the same rank?

(c) Let $A =$ "first card is an ace" and $B =$ "second card is an ace". Are $A$ and $B$ independent? Justify your answer.

(d) Given that the first card drawn is an ace, what is the probability that the second card is also an ace?

**Problem 2: Linearity of Expectation.**

(a) A fair die is rolled 10 times. Let $X$ be the total sum. Compute $\mathbb{E}[X]$.

(b) A deck of 52 cards is shuffled uniformly at random. A card is a **match** if it lands in position $i$ and has rank $i \bmod 13$. Let $X$ be the number of matches. Compute $\mathbb{E}[X]$.
*Hint: use indicator random variables.*

(c) You roll a fair die repeatedly until you see a 6. Let $X_i = 1$ if roll $i$ is not a 6, and 0 otherwise. Explain why computing $\mathbb{E}$[number of non-6 rolls before the first 6] via linearity of expectation is *not* straightforward here, and state what additional tool would be needed.

## Bloom Filters

**Problem 3: Implementing a Bloom Filter.**
Generate the dataset $S$ as follows:

```
import numpy as np
rng = np.random.default_rng(42)

lo = rng.integers(0,       500_000, size=500_000)
hi = rng.integers(1_000_000, 1_500_000, size=500_000)
S  = set(np.concatenate([lo, hi]).tolist())   # ~1M inserted elements
```

Using the parameters derived in lecture ($k = \lceil \log_2(1/\varepsilon) \rceil$, $m = \lceil n \log_2(1/\varepsilon) / \ln 2 \rceil$), implement a Bloom filter and answer the following.

Use the following hash family:

```
def _mix(x, seed, m):
    x = (x ^ seed) & 0xffffffff
    x = ((x ^ (x >> 16)) * 0x45d9f3b) & 0xffffffff
    x = ((x ^ (x >> 16)) * 0x45d9f3b) & 0xffffffff
    return ((x ^ (x >> 16)) & 0xffffffff) % m

def hashes(x, k, m):
    return [_mix(x, i * 0x9e3779b9, m) for i in range(k)]
```

To get different hash functions, use different seeds $k$.

(a) Implement `BloomFilter(n, eps)` that supports `insert(x)` and `query(x)` using `hashes` above.

(b) Insert all elements of $S$ into the filter. Verify there are **no false negatives**: every element of $S$ must return `True` on `query`.

(c) Generate $10^5$ test integers uniformly at random from $[500\,001, 999\,999]$ (the gap between the two inserted ranges — guaranteed not in $S$). Query each one and report the **empirical false positive rate**.

(d) Repeat part (c) for $\varepsilon \in \{0.10, 0.05, 0.01, 0.001\}$. For each setting, record the theoretical $\varepsilon$, the empirical rate, and the values of $k$ and $m$ used. Present your results in a table.

(e) Does doubling $m/n$ roughly square the false positive rate in your experiments? Show the relevant rows from your table and comment.

**Problem 4: Bloom Filter with Deletions.**

A standard Bloom filter cannot support deletions: clearing a bit when removing $x$ might destroy information about other inserted elements that hash to the same position.

(a) Explain with a concrete example why clearing a bit on deletion is incorrect. Use $k = 2$, $m = 8$, and two elements $x$, $y$ that share a hash position.

(b) Explain how to modify vanilla Bloom filters to handle deletions.